

LMCP AutoQuote System Build Channel

Current Status Snapshot

What is already in place

- Core FastAPI application structure exists.
- Docker environment has been used and can run, but stability is inconsistent.
- Main application entry files and several API/service modules already exist.
- Supply-and-delivery focus has been defined.
- Profitability rules are defined at a high level:
 - South Africa only
 - Supply and delivery only
 - Exclude medical consumables, IT equipment, petrol and diesel
 - Exclude tenders with briefing sessions
- Estimated minimum profit target: R30,000+
- Default margin assumption: at least 25%

What is partially built but not yet fully reliable

- Tender pipeline routing
- Live RFQ storage and filtered output logic
- Harvester-to-pipeline integration
- Auto-score and filtering flow
- Quote trigger flow
- Swagger/docs visibility of all expected endpoints

Main blockers seen so far

- Conflicting or duplicated models/files
- Router imports not consistently wired into `app/main.py`
- Some files are longer than patch snippets, causing misplacement of code
- Indentation and structure errors from partial manual edits
- Endpoint availability changes between runs
- Docker/app restarts sometimes leave the system in an unclear state

Honest position

The system is **partially built**, but **not yet stable enough for production or unattended execution**. The architecture is there, but wiring and cleanup are still needed.

Correct Build Order From Here

Phase 1 — Stabilise the foundation

1. Confirm project root and active codebase
2. Freeze and inspect `app/main.py`
3. Freeze and inspect active routers
4. Remove duplicate/conflicting model and schema definitions
5. Confirm Docker services and app startup path
6. Restore stable `/docs`

Phase 2 — Lock the tender pipeline

1. Finalise `tender_pipeline_api.py`
2. Finalise `tender_pipeline.py`
3. Finalise `live_rfqs_store.py`
4. Confirm `/tender-pipeline/*` endpoints appear and work
5. Confirm `/supply-command/live-rfqs` appears and works

Phase 3 — Business rules and filtering

1. Enforce supply-and-delivery-only targeting
2. Exclude forbidden categories
3. Exclude briefing-session tenders
4. Apply minimum estimated profit logic
5. Apply 25% default margin assumption
6. Save only qualified opportunities

Phase 4 — Automation flow

1. Harvester writes RFQs into live store
2. Pipeline scores each RFQ
3. Eligible RFQs trigger quote engine
4. Compliance pack attaches correctly
5. Results are visible in API output

Phase 5 — Testing and hardening

1. Endpoint-by-endpoint tests
 2. Real sample RFQ tests
 3. Failure logging cleanup
 4. Docker restart test
 5. Backup snapshot
-

Single Working Method To Finish Faster

Rule 1

We work in **one build thread only** until the system is stable.

Rule 2

We change **one file at a time**.

Rule 3

For any file edit, we use this order: 1. identify exact active file 2. show full current file if needed 3. replace with full clean code 4. run app 5. verify endpoint/result 6. only then move to next file

Rule 4

No partial snippets for critical files.

Rule 5

Every completed file gets marked done before moving on.

Master File Priority List

1. `app/main.py`
2. `app/api/tender_pipeline_api.py`
3. `app/api/supply_command_api.py`
4. `app/services/tender_pipeline.py`
5. `app/services/live_rfq_store.py`
6. `app/services/tender_eligibility.py`
7. `app/tasks/...` active task file(s)
8. duplicate model/schema cleanup
9. quote engine trigger path

Definition of Done

The system is only “finished” when all of the following are true: - `/docs` loads consistently - expected routers appear consistently - harvester can write RFQs - filtering rules work correctly - only profitable supply-and-delivery RFQs survive - quote trigger runs automatically for qualified RFQs - Docker restart does not break the app - backup of stable version is created

Next Execution Step

Start with **Phase 1, File 1:** `app/main.py` and cleanly lock the active router wiring before touching anything else.